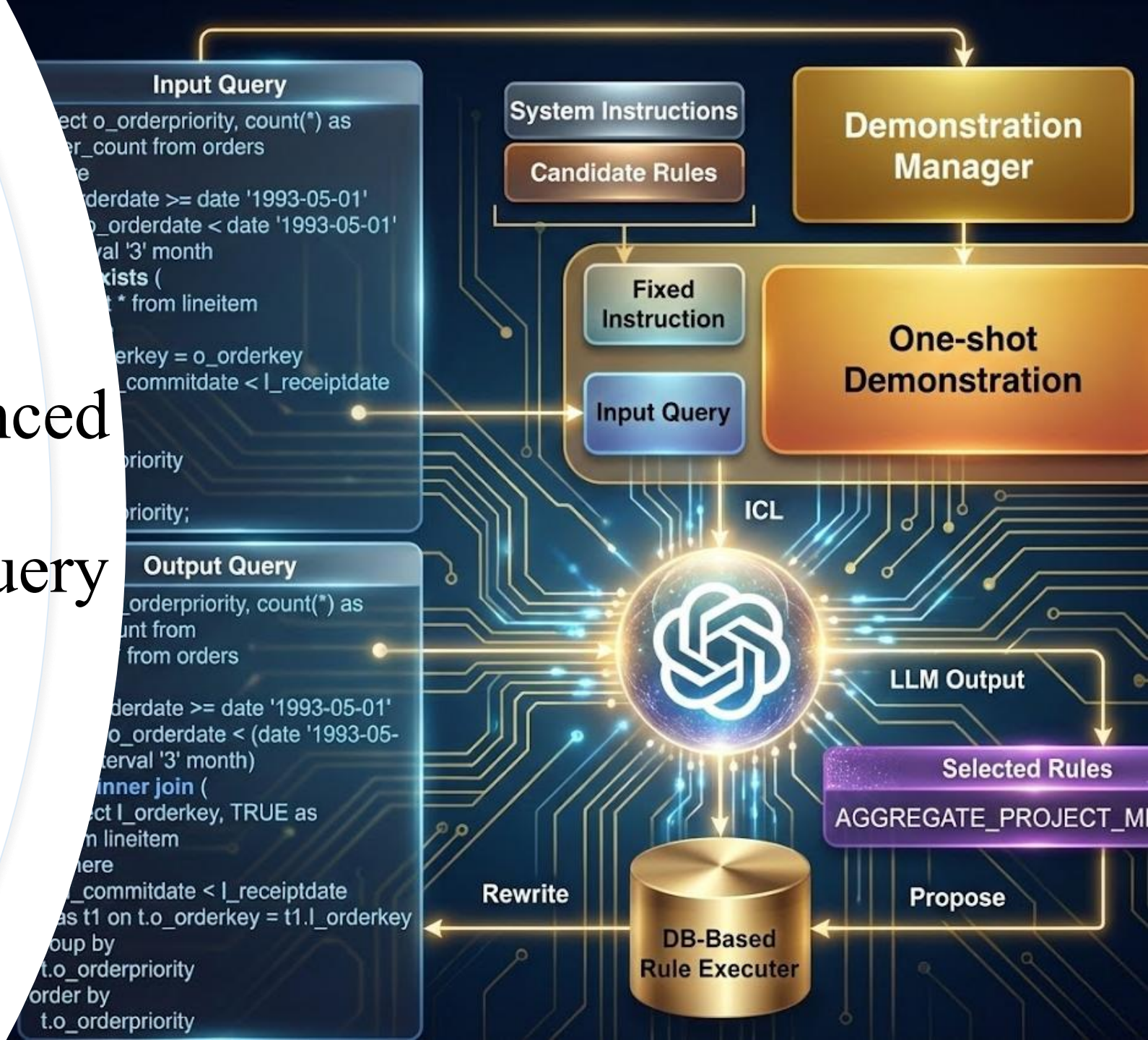
LLM-R2: A Large Language Model Enhanced Rule-based Rewrite System for Boosting Query Eficiency

Altan Ulaş Zöhre

Deniz Sakaroğlu

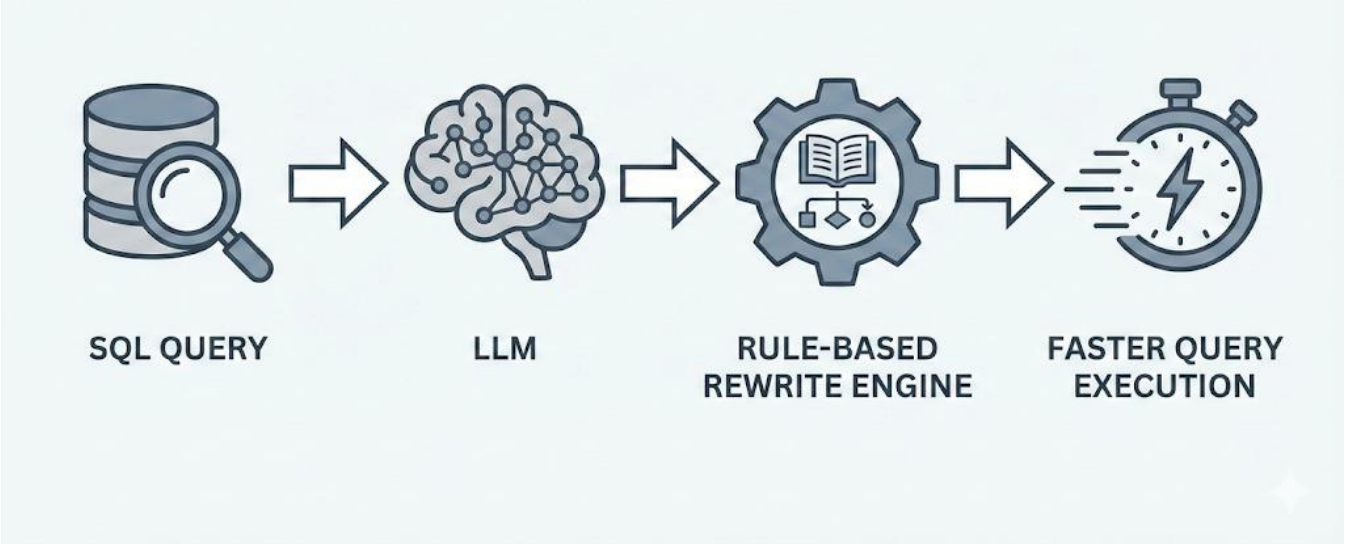
CENG 544 Large-Scale Data Management

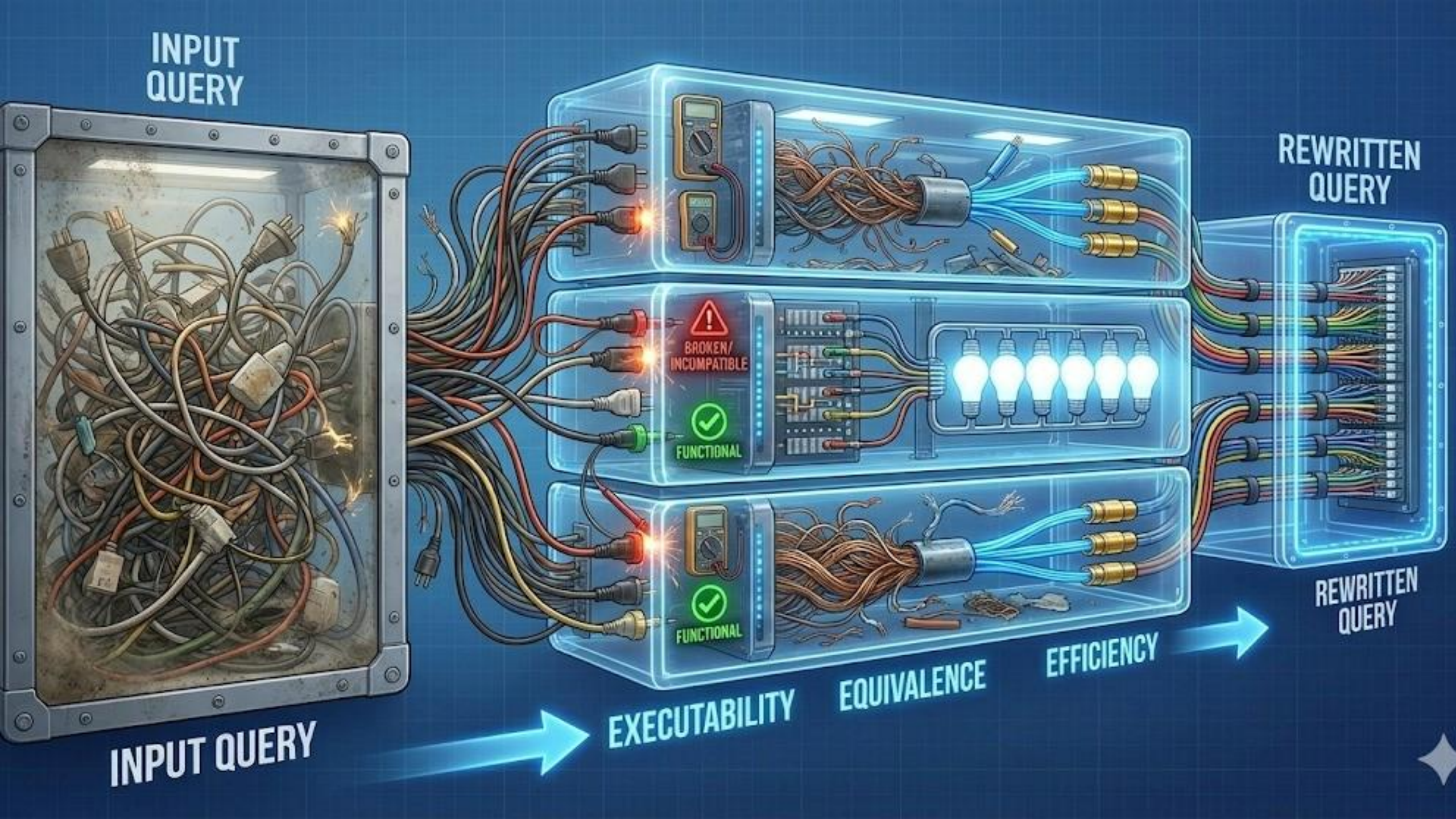
31.12.2025



What is Query Rewriting

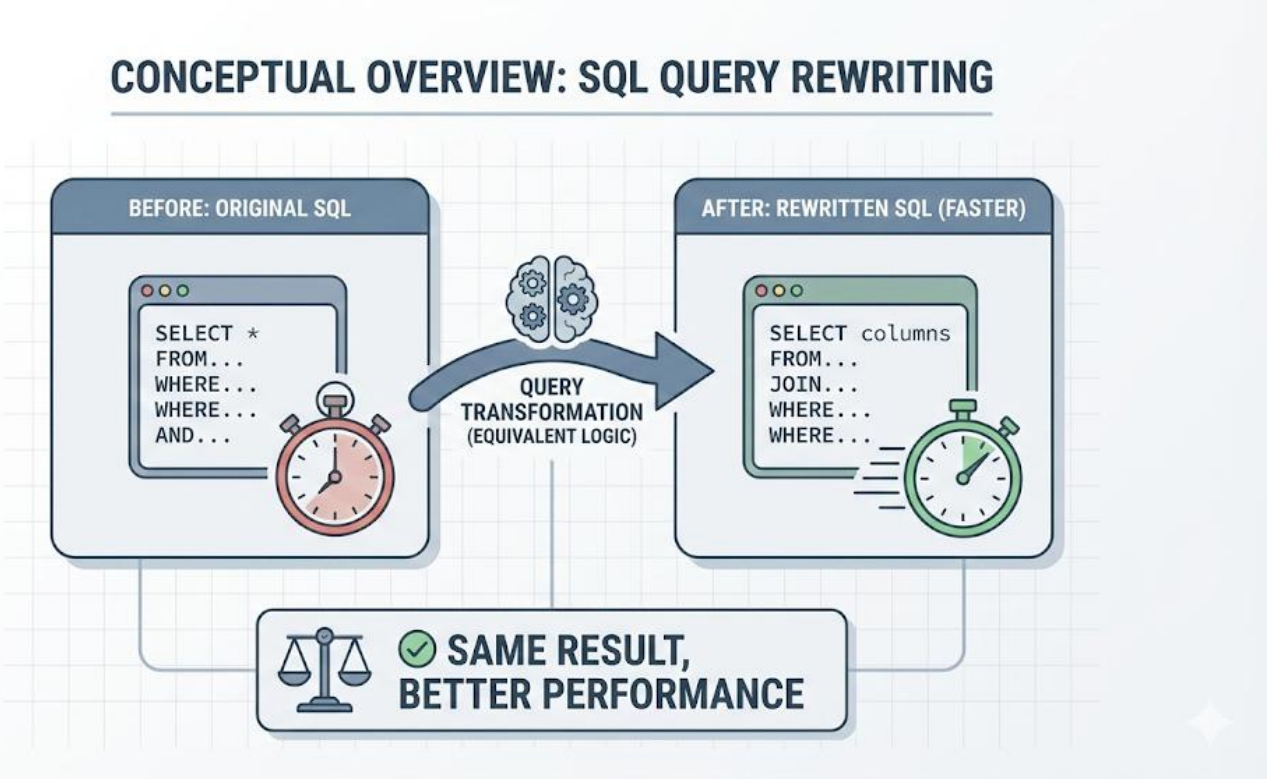
- Query rewrite aims to improve query efficiency by altering an SQL query's structure without changing its result.
- The objective is to generate an equivalent query that executes in less time.
- It must satisfy three critical criteria:
  - **Executability:** The rewritten query must run without errors.
  - **Equivalence:** It must produce identical results as the original query.
  - **Efficiency:** The rewrite must improve execution time significantly.

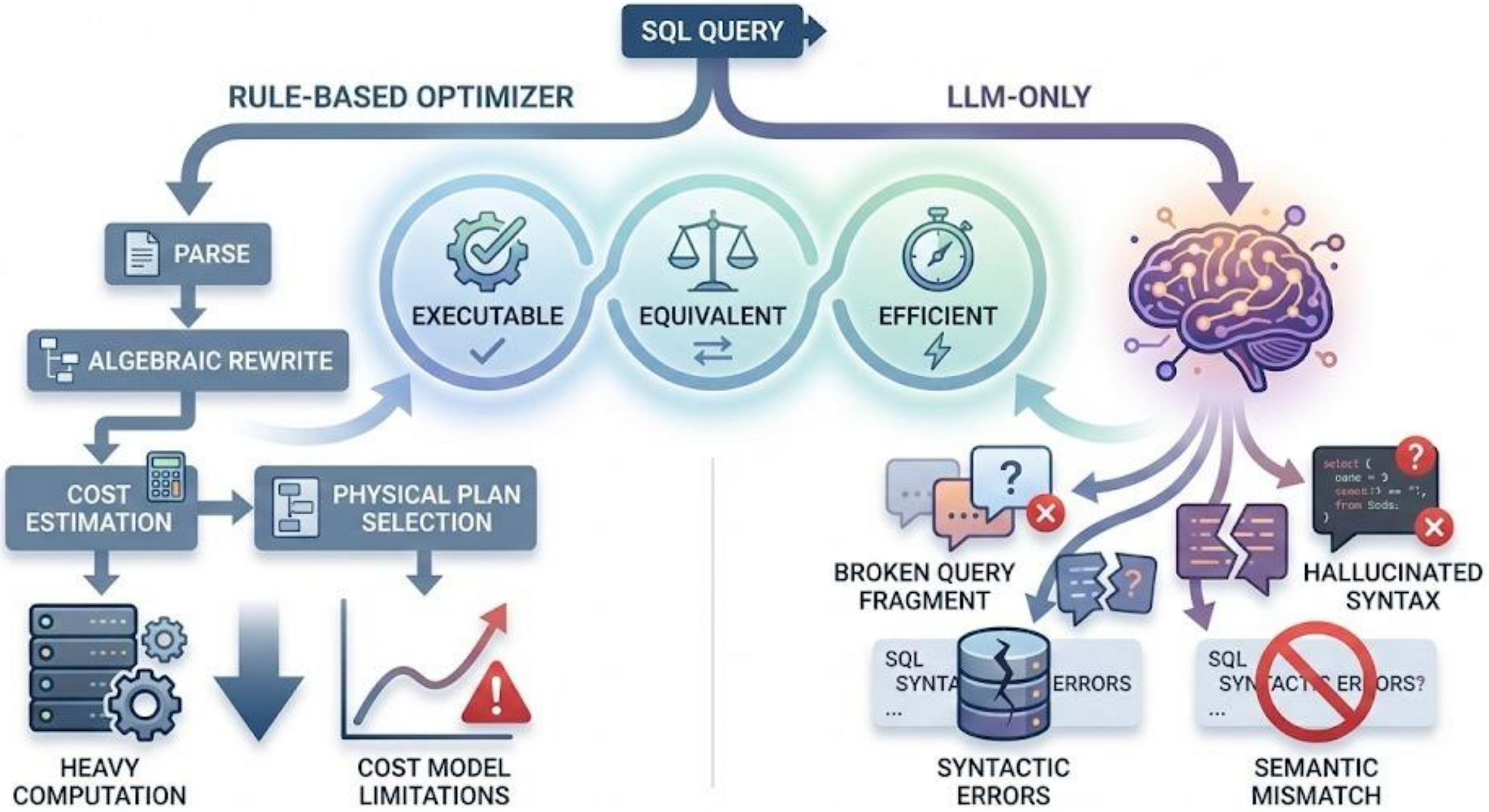



INPUT QUERY
BROKEN/ INCOMPATIBLE
FUNCTIONAL
FUNCTIONAL
REWRITTEN QUERY
INPUT QUERY
EXECUTABILITY
EQUIVALENCE
EFFICIENCY
REWRITTEN QUERY

Motivation and Challenges

- Existing methods for finding the optimal sequence of rewrite rules are often limited and resource-intensive.
- Discovering new rules typically requires complex proofs or extensive user interactions.
- Current systems rely heavily on DBMS cost estimators, which are frequently inaccurate.
- Direct rewriting by Large Language Models (LLMs) suffers from "hallucinations," leading to incorrect or unexecutable SQL code.





Query rewrite must satisfy all three simultaneously

The Proposed Solution: LLM-R2

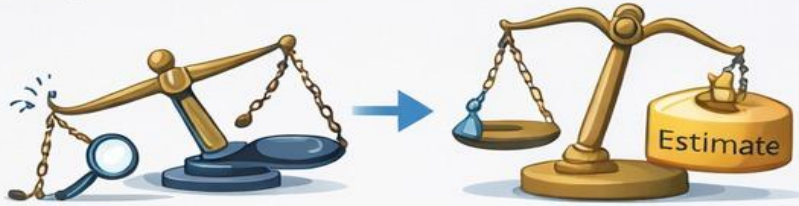
- The authors propose **LLM-R2**, a novel method that leverages an LLM to recommend rewrite rules for a database system.
- This system integrates the generalization and reasoning abilities of LLMs while avoiding the risk of hallucination.
- By using LLMs to suggest rules from pre-validated database platforms, the system guarantees the executability and equivalence of the rewritten query.

Why Existing Approaches Fall Short

1) Traditional Rule-Based Rewrite

1. Searching for a good sequence of rewrite rules (can be expensive)

2. Inaccurate DB Cost Estimator
→ Suboptimal Choices



3. Inaccurate DB Cost Estimator
→ Suboptimal Choices

2) LLM-Only Rewrite (Generate SQL Directly)

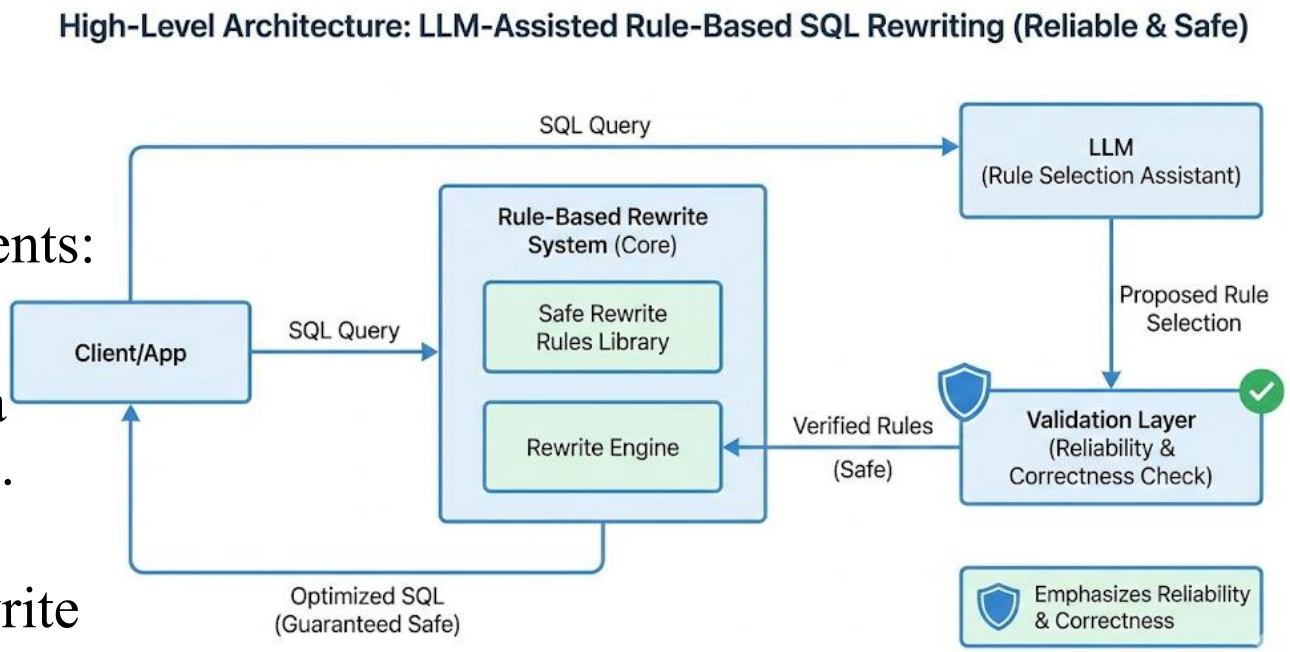
1. Produces Invalid SQL
→ Not Executable

2. Breaks Semantic Equivalence (Hallucination)
→ Incorrect Results

2. Hard to Guarantee Correctness

System Pipeline Overview

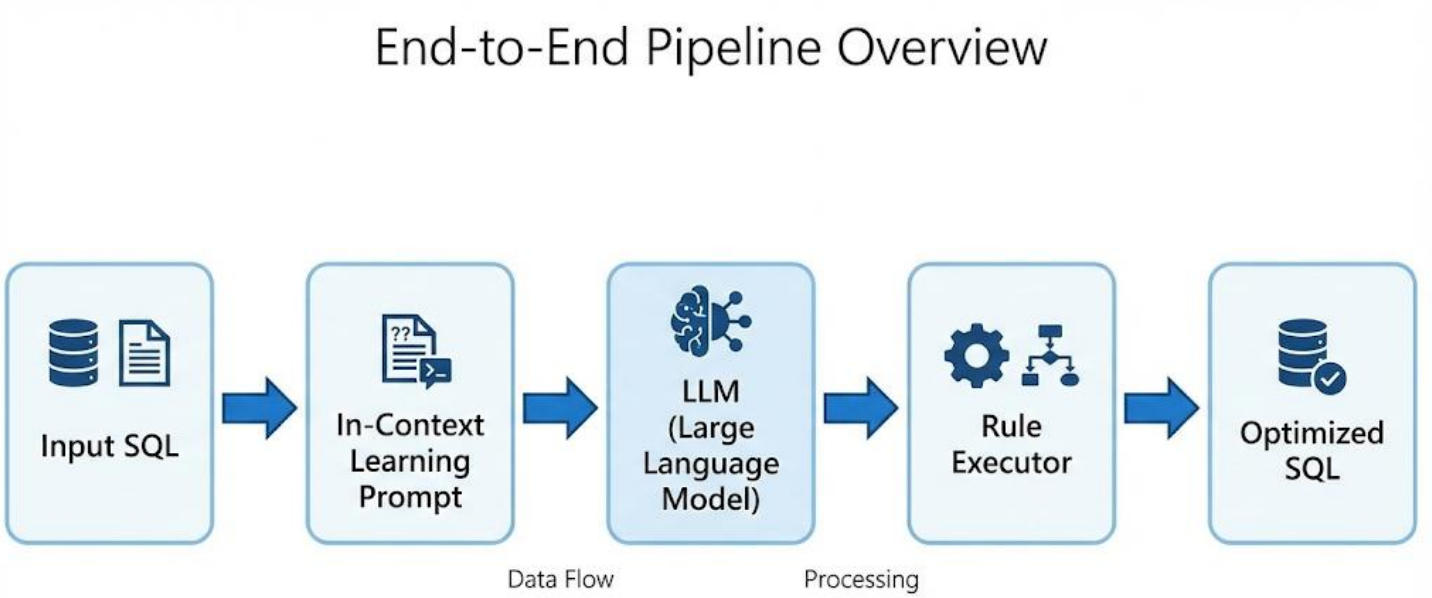
- The system integrates an LLM into the rewrite process using In-Context Learning (ICL).
- The ICL prompt consists of three main components:
  - **Input Query:** The original SQL statement.
  - **Fixed Instruction:** Task requirements and a comprehensive list of candidate rewrite rules.
  - **One-shot Demonstration:** A high-quality example query paired with its successful rewrite rules to guide the model.
- Rules suggested by the LLM are processed and executed by a database-based rule executor.

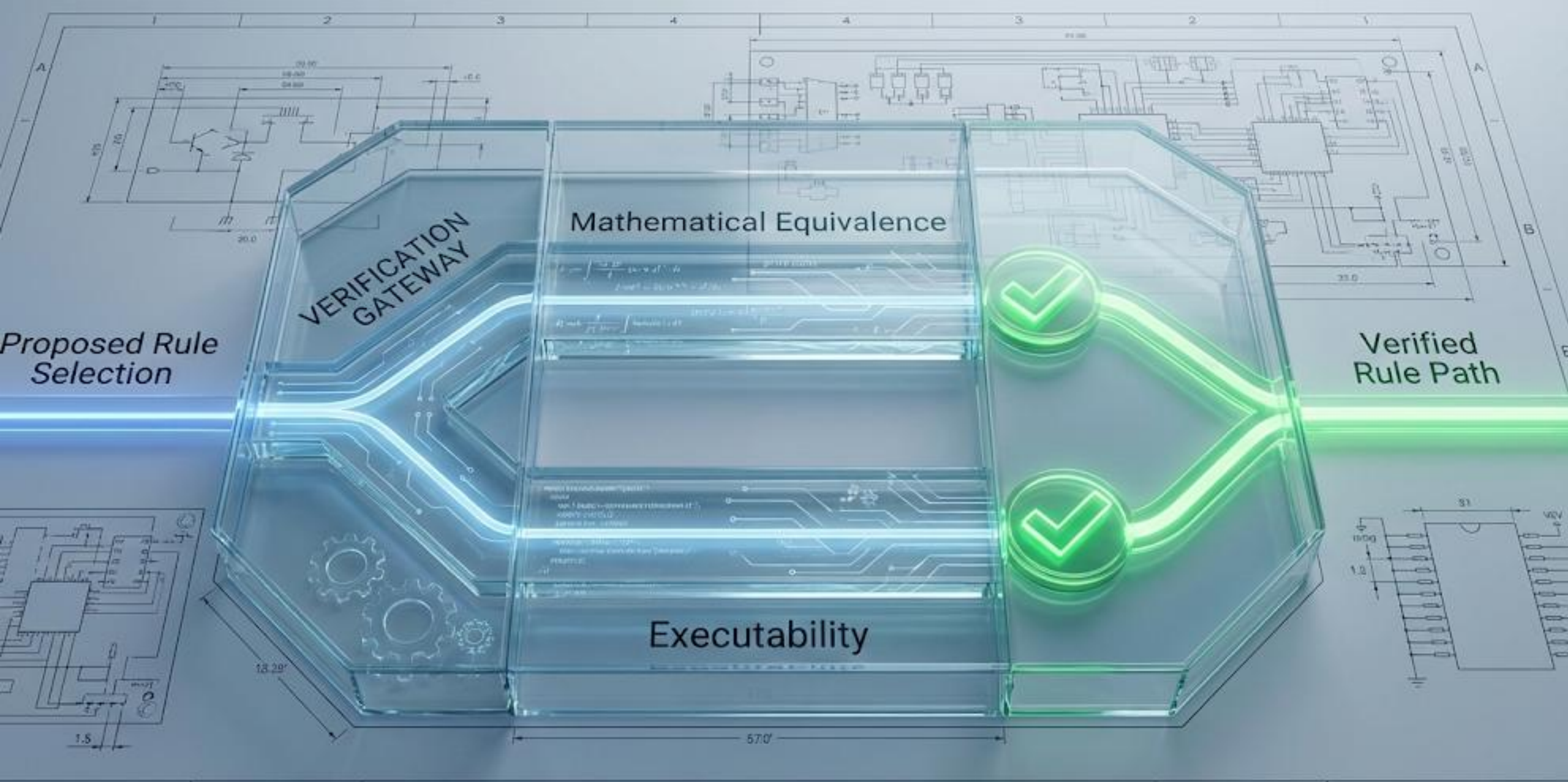


DATABASE OPTIMIZATION SYSTEM

Preventing Hallucination

- Relying solely on an LLM's output query can violate basic requirements like equivalence and executability.
- To overcome this, LLM-R2 restricts the LLM's role to selecting rules from a given set provided by existing platforms like Apache Calcite.
- All candidate rules are pre-validated, ensuring that the final rewritten query is mathematically equivalent to the original.





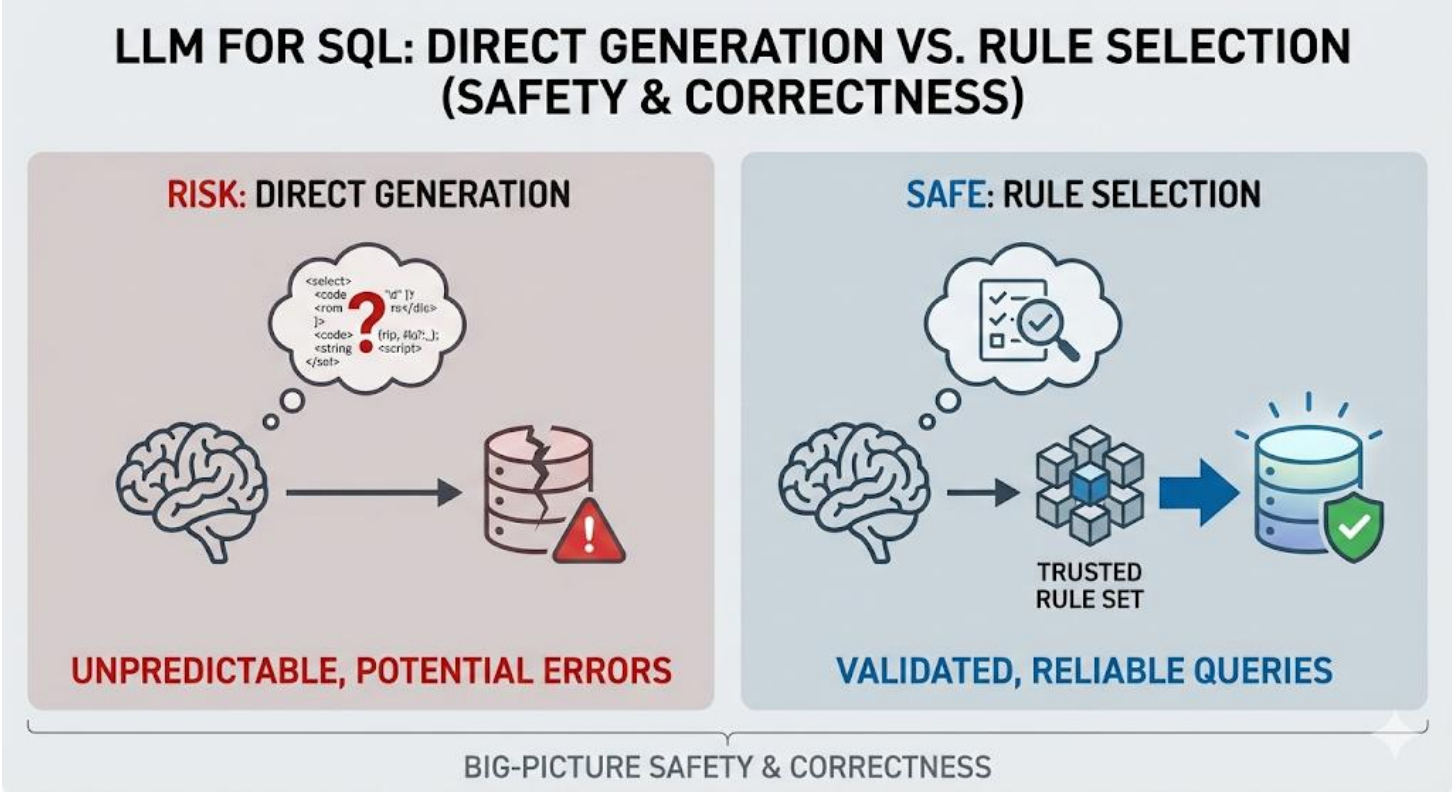
ENGINEERING BLUEPRINT: RULE VERIFICATION SYSTEM

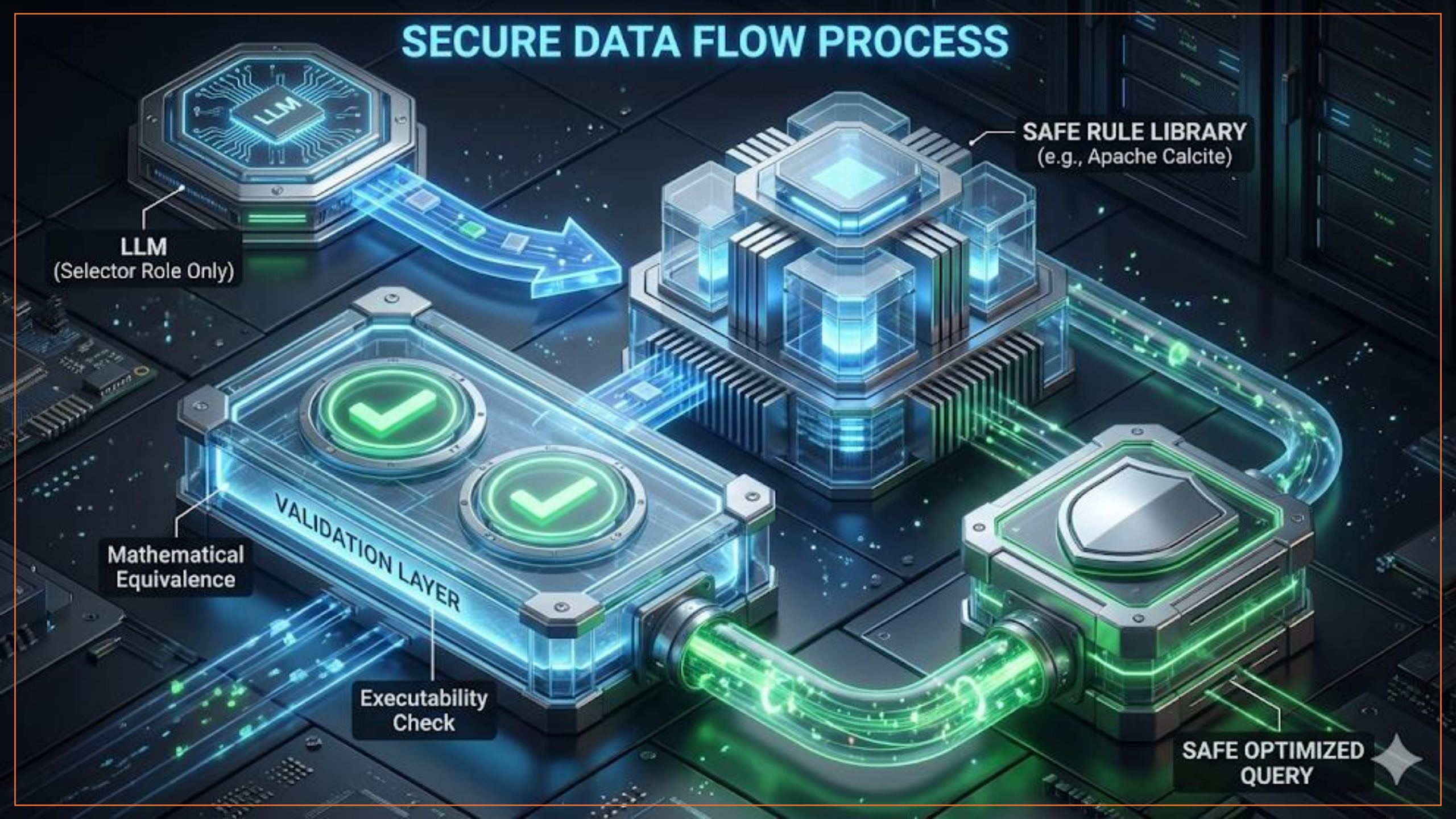
TITLE

- REV 1.0 - APPROVED FOR DEPLOYMENT

Introduction to Demonstration Manager

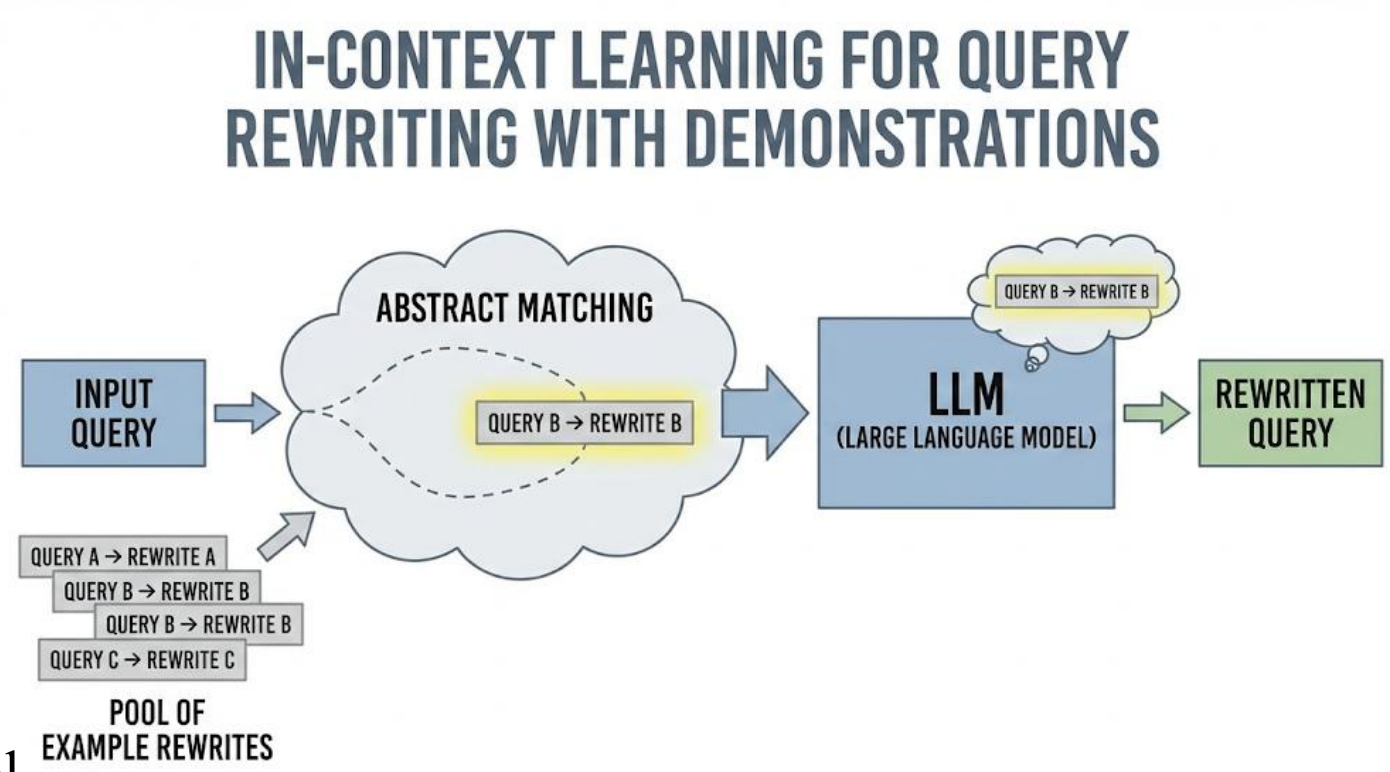
- Optimizing the demonstration in the prompt is crucial for improving LLM output quality.
- The **Demonstration Manager** is designed to identify the best examples to guide the LLM for any given input query.
- It consists of two primary modules:
  - **Demonstration Preparation:** Generating a pool of successful rewrite cases.
  - **Demonstration Selection:** Matching the most appropriate example to a new input query in real-time.




SECURE DATA FLOW PROCESS
LLM
LLM
(Selector Role Only)
SAFE RULE LIBRARY
(e.g., Apache Calcite)
VALIDATION LAYER
Mathematical
Equivalence
Executability
Check
SAFE OPTIMIZED
QUERY

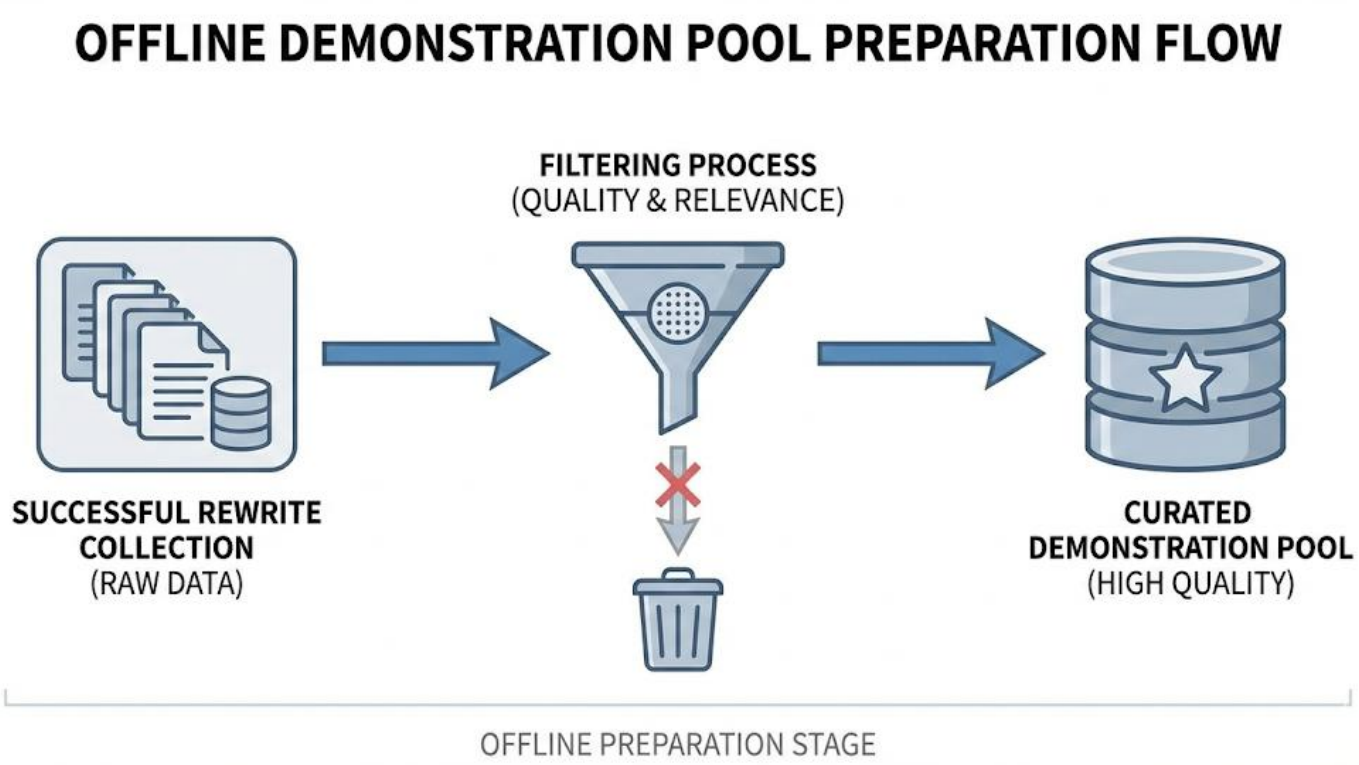
Phase 1: Demonstration Preparation

- The goal is to build a pool of successful rewritten demonstrations and supply training data.
- **Benefit Estimator:** Assesses the potential benefits of rewrite strategies and generates performance records.
- **Pool Generator:** Extracts the best samples to construct a high-quality demonstration pool for online inference.
- Training triplets are created to help teach the selection model how to distinguish between "improving" and "degrading" rewrites.



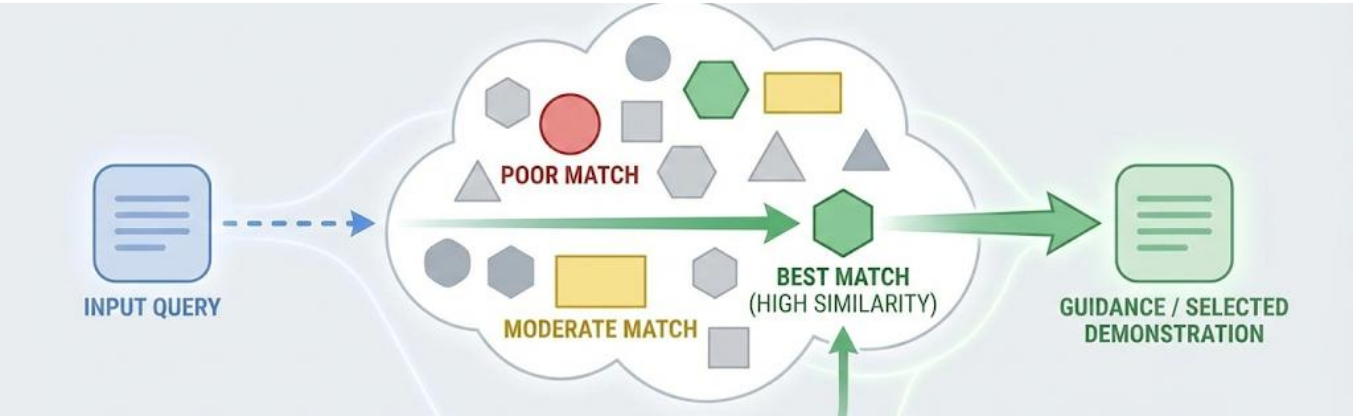
Phase 2: Demonstration Selection

- This module identifies the optimal demonstration from the pool for each new input query.
- It uses a **query representation model** within a selector to evaluate similarity between input queries and stored demonstrations.
- The model is trained offline using contrastive learning techniques to maximize its selection accuracy.



Query Representation Model

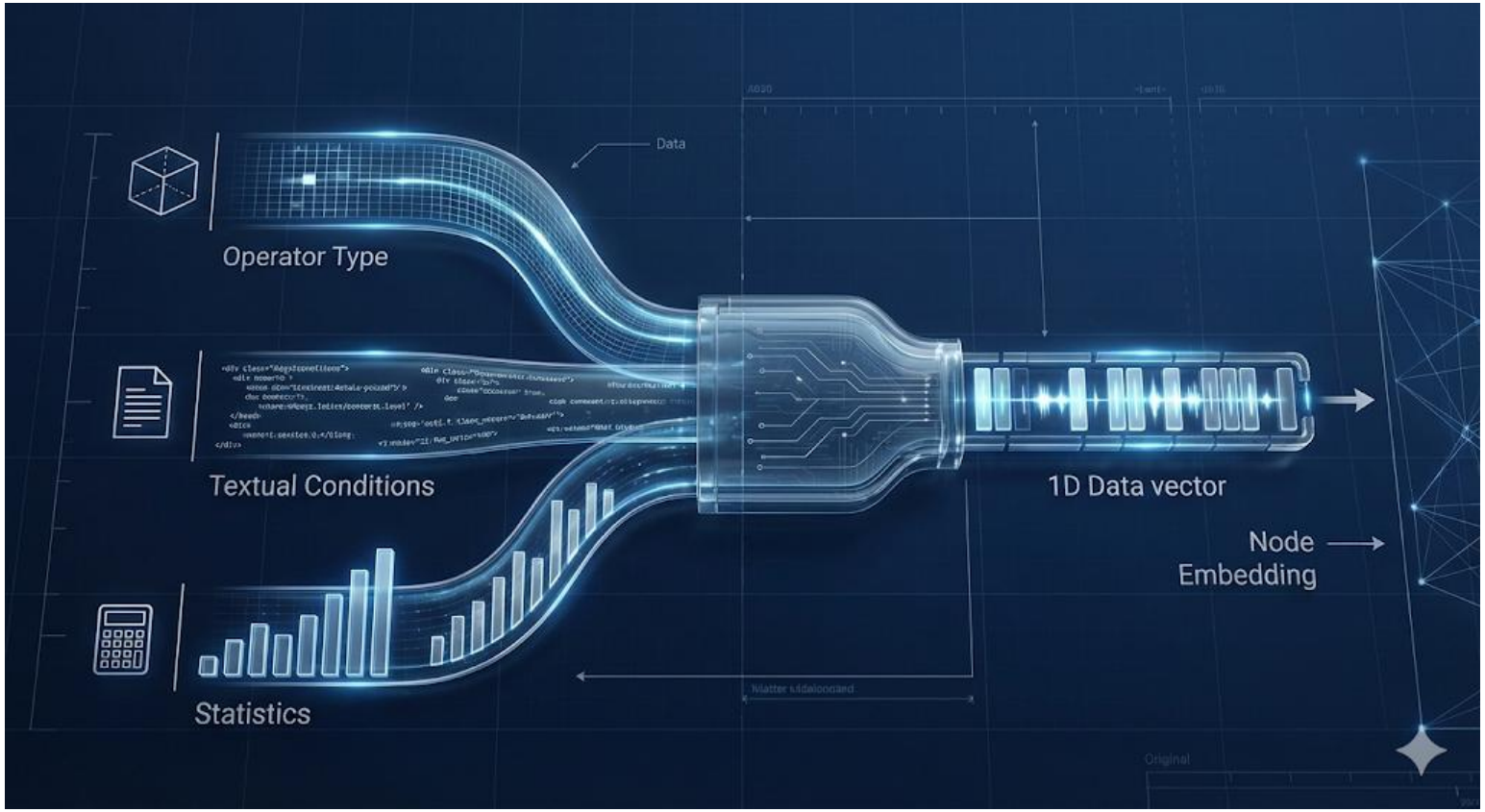
- The system encodes queries using trees where nodes represent specific operators like Sort, Join, and Scan.
- The model transforms each query tree node into a fixed-length vector.
- A **tree-biased attention layer** is used to compute the final representation of the entire query.
- This approach captures structural and statistical attributes to better understand query similarity.



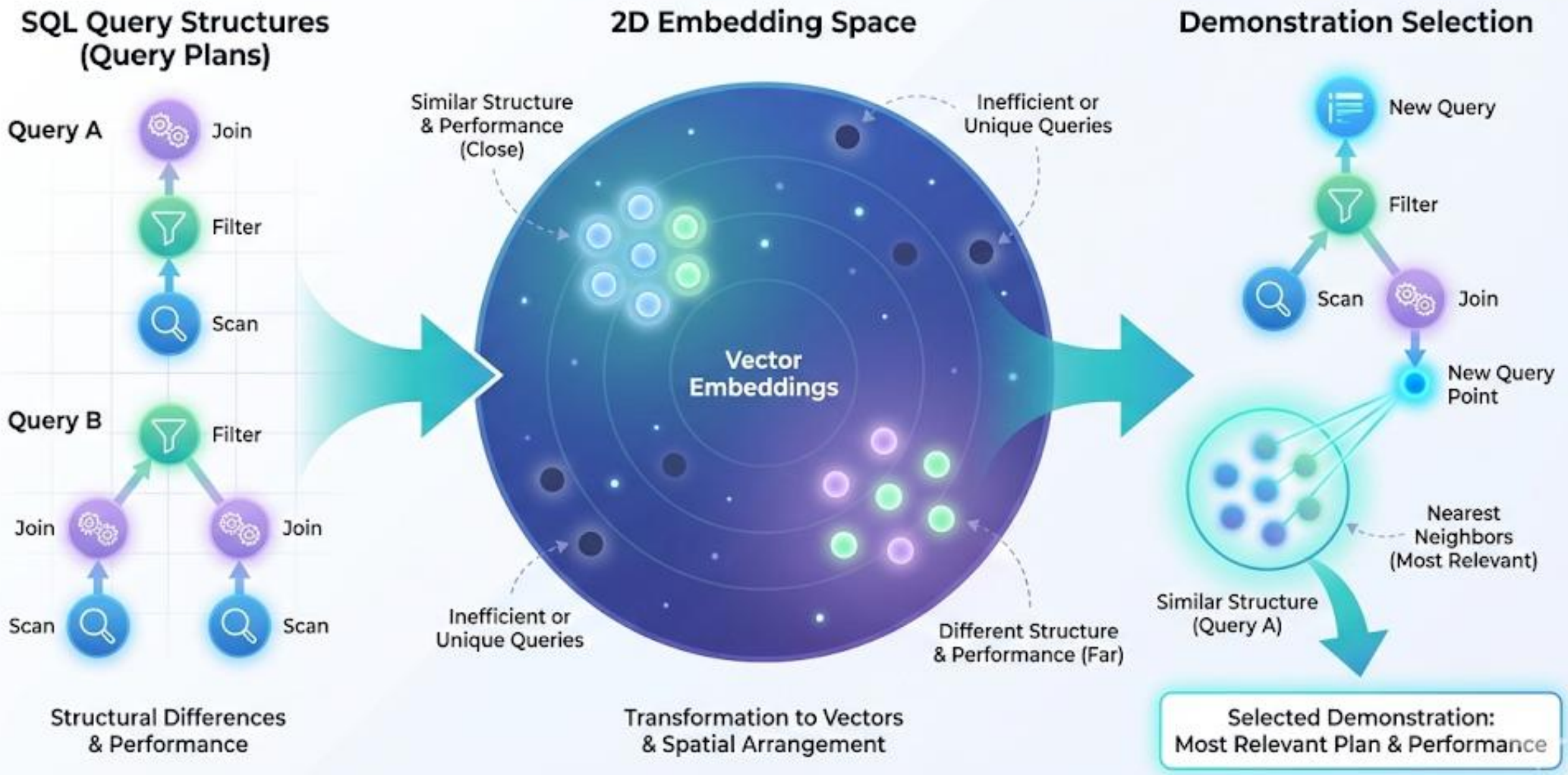
Feature Encoding Details

Each node embedding includes three types of information:

- **Operator Type:** One-hot encoding for the specific operator.
- **Operator Conditions:** Textual details like filter criteria, encoded using Sentence Transformers.
- **Statistics:** Estimated cardinality (row counts) and cumulative costs, normalized through an MLP layer.

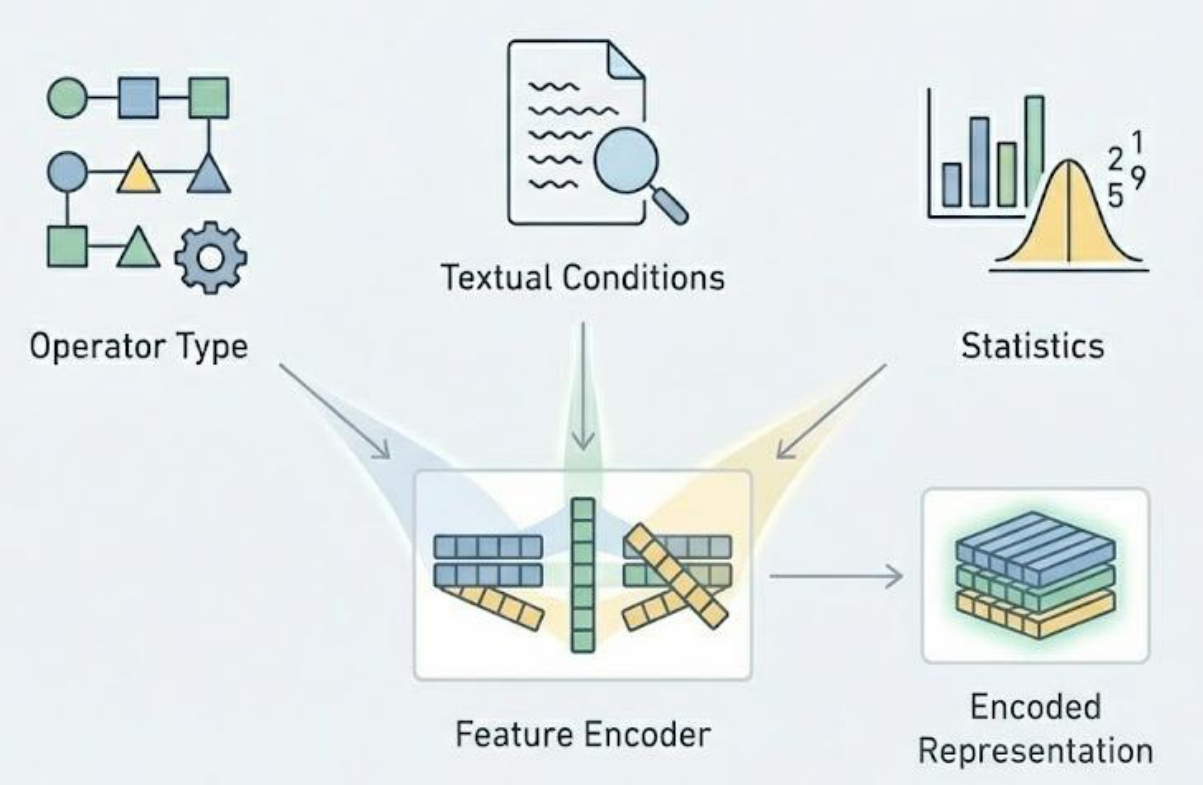


QUERY REPRESENTATION AND EMBEDDING SPACE

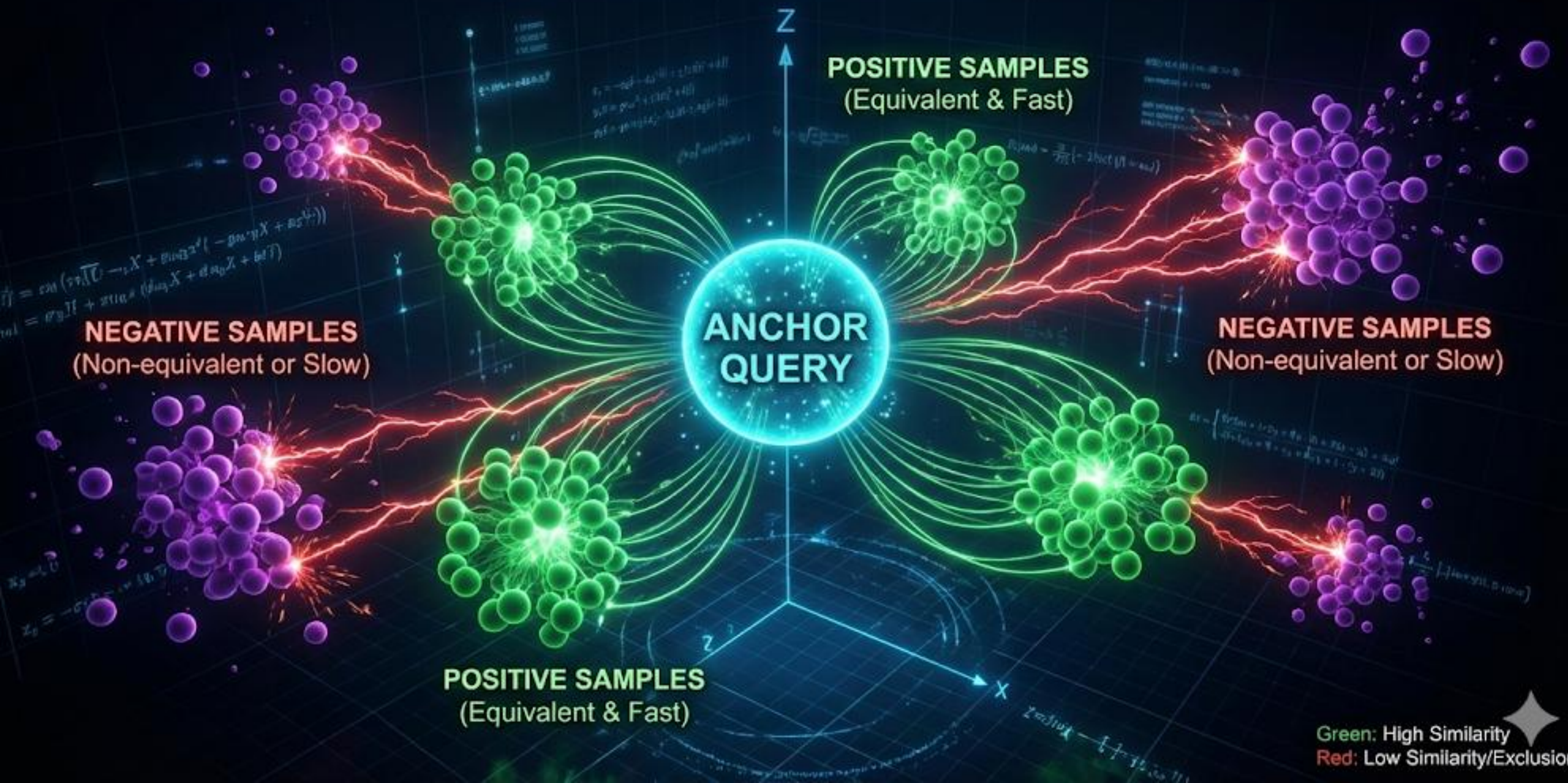


Training Strategy: Contrastive Learning

- To capture query representations effectively with limited data, the authors use a contrastive learning structure.
- For each query, the model considers:
  - **Positive samples:** Versions that improved performance.
  - **Hard Negative samples:** Versions that degraded performance.
- The training objective is to pull original queries closer to their improved versions in the vector space while pushing away degraded ones.

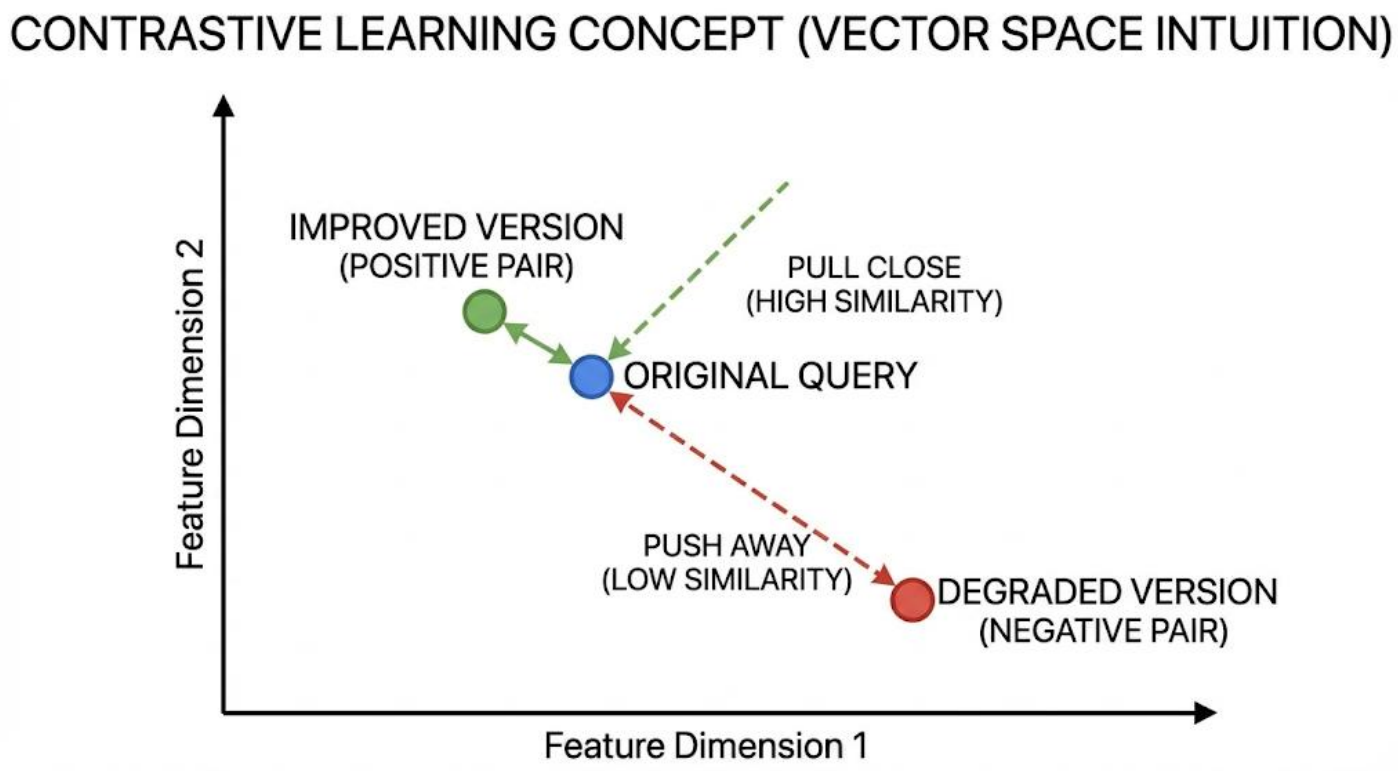


QUERY REPRESENTATIONS IN VECTOR SPACE



Addressing Data Scarcity: Curriculum Learning

- High execution costs for queries lead to scalability issues and limited training data.
- The authors propose a **curriculum learning pipeline** to enhance learning efficiency.
- This strategy organize the training data from easy to hard, starting with a simpler subset and progressively incorporating more complex cases.



CURRICULUM LEARNING HIERARCHY

PROGRESSION:
Complex to Mastery

Top Layer: Complex Queries

CTEs (Common Table Expressions)

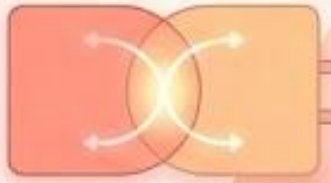
RANK()

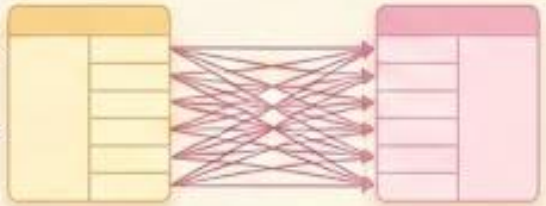
SUM() OVER()

ROW_NUMBER()

PROGRESSION:
Structured to Complex

Middle Layer: Join Operations





PROGRESSION:
Simple to Structured

Base Layer: Basic SQL Operations

FILTER FUNNELS

SCAN & SELECT

Experimental Setup

- **Datasets:** TPC-H (10GB), IMDB (Join Order Benchmark), and Decision Support Benchmark (DSB).
- **Rewrite Platform:** Apache Calcite with its comprehensive set of rewrite rules.
- **LLM Setting:** Default use of GPT-3.5-turbo, with additional tests on GPT-4, Llama3, and Granite.
- **Baselines:** Compared against **Learned Rewrite (LR)** and a direct **LLM only** approach.

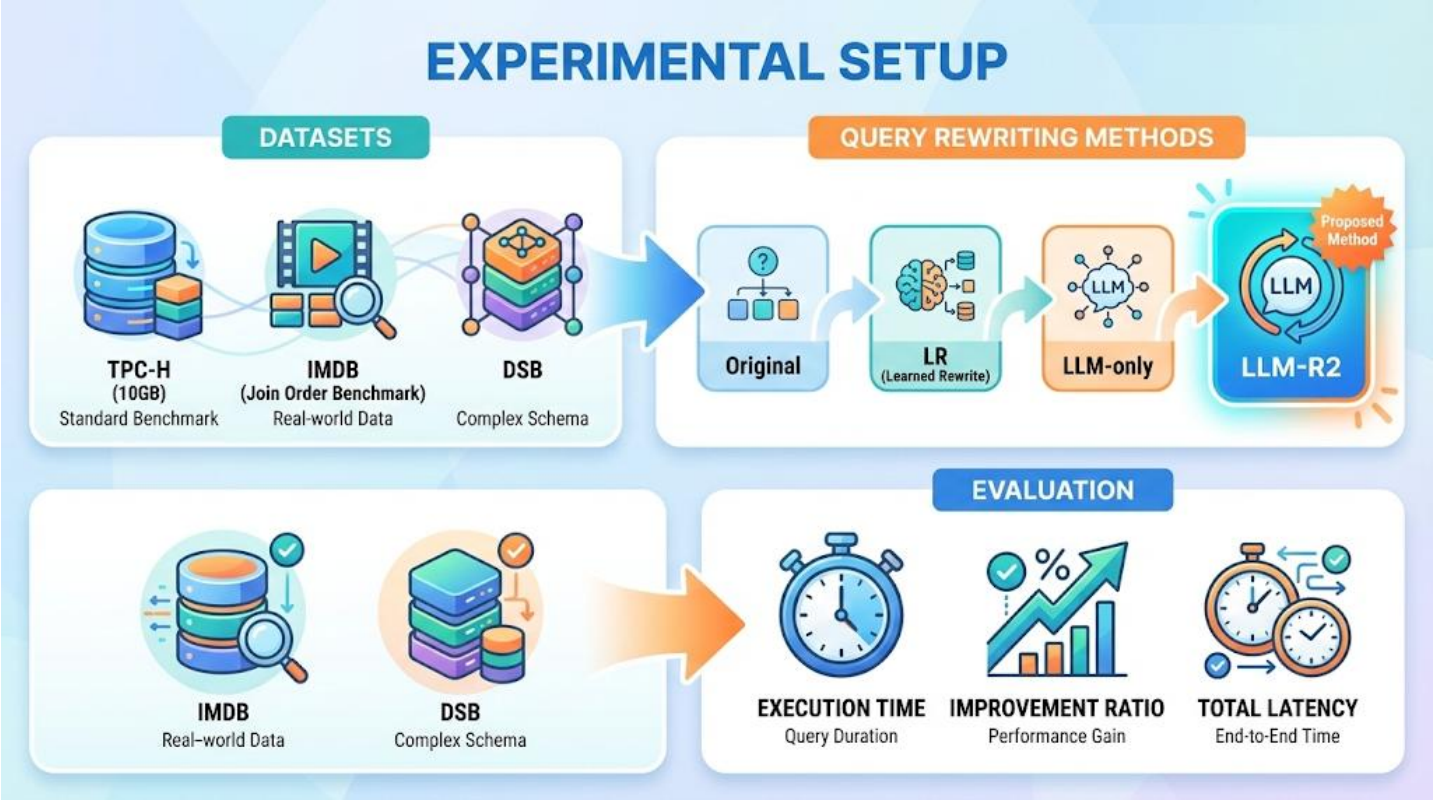


Table 2: Execution time v.s. different query rewrite methods

Execution time(sec)	TPC-H				IMDB				DSB			
Method	Mean	Median	75th	95th	Mean	Median	75th	95th	Mean	Median	75th	95th
Original	70.90	22.00	37.01	300.00	6.99	1.86	5.12	32.49	60.55	6.64	26.55	300.00
LR	39.40	22.00	32.21	**159.95**	6.20	1.62	4.74	32.45	59.21	5.14	53.78	300.00
LLM only (GPT-3.5)	70.67	22.00	37.01	300.00	6.96	1.86	5.10	32.49	61.60	6.53	26.40	300.00
LLM-R^2 (GPT-3.5)	**37.23**	**17.40**	**29.80**	164.12	**3.91**	**1.33**	**3.52**	**18.16**	**24.11**	**2.16**	**12.61**	**196.61**
% of **Original**	52.5%	79.1%	80.5%	54.7%	56.0%	71.3%	68.7%	55.9%	39.8%	32.5%	47.5%	65.5%
% of **LR**	94.5%	79.1%	92.5%	102.6%	63.1%	82.0%	74.3%	56.0%	40.7%	42.0%	23.4%	65.5%
% of **LLM only**	52.7%	79.1%	80.5%	54.7%	56.2%	71.3%	69.0%	55.9%	39.1%	33.1%	47.8%	65.5%
LLM only (Llama3)	70.80	22.00	37.01	300.00	6.98	1.86	4.99	32.49	62.02	6.62	26.55	300.00
LLM only (Granite)	65.06	20.73	36.25	300.00	6.03	1.85	4.91	32.49	60.55	6.64	26.55	300.00
LLM-R^2 (Llama3)	38.47	**18.99**	31.55	161.03	5.94	1.83	4.88	29.69	51.35	6.12	27.07	**300.00**
LLM-R^2 (Granite)	**37.89**	19.75	**28.62**	**157.37**	**4.71**	**0.97**	**3.65**	**21.56**	**26.58**	**3.46**	**13.15**	**300.00**

Table 3: The rewritten queries' number

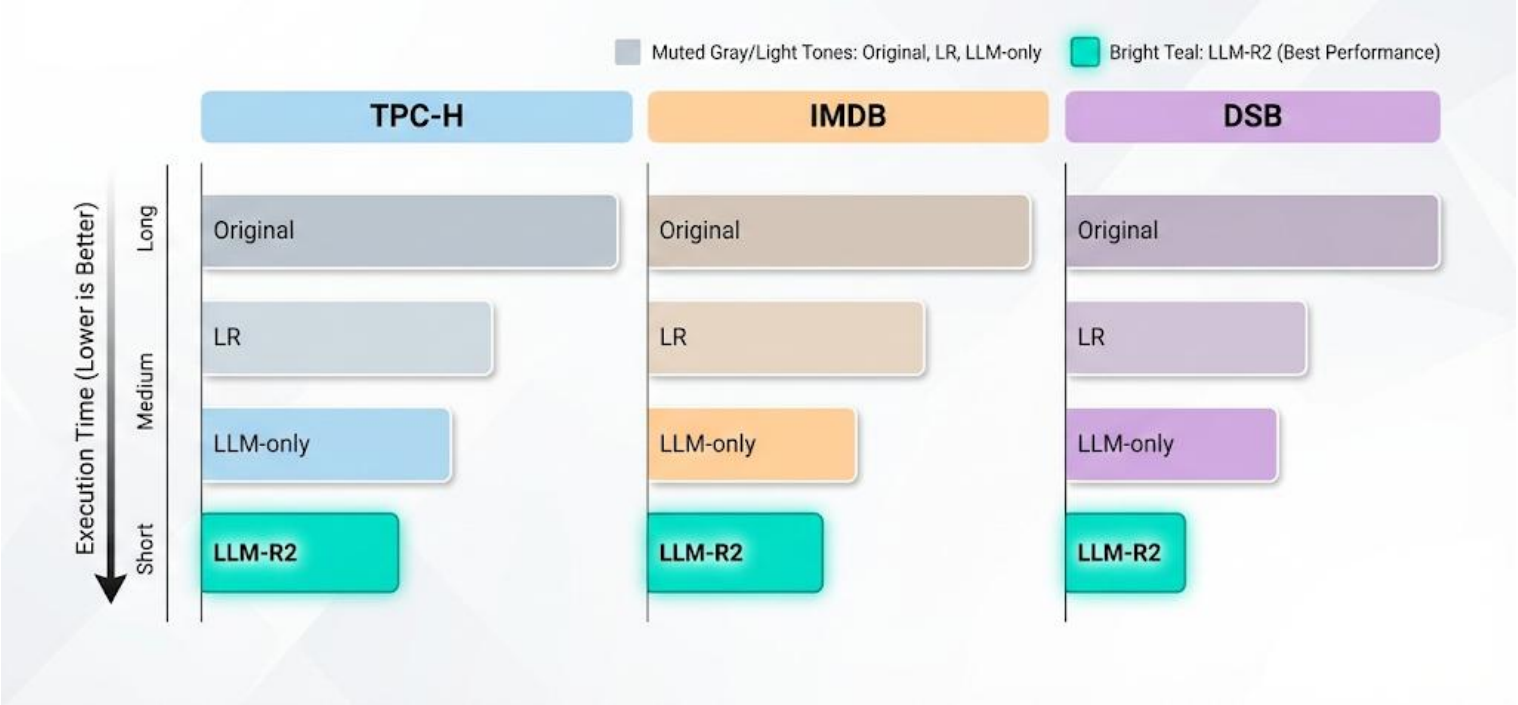
Counts	TPC-H/IMDB/DSB		
Method	Rewrite #	Improve #	Improve %
LR	258/203/**456**	192/197/193	74.42/**97.04**/42.32
LLM only	197/102/210	68/67/8	34.5/65.68/3.81
LLM-R^2	**323**/**302**/341	**305**/**292**/**222**	**94.43**/96.69/**65.10**

Table 4: Query execution time including the rewrite latency

Total (Latency)	TPC-H	IMDB	DSB
LR	40.98(1.58)	7.24(1.04)	60.99(1.78)
LLM only	75.37(4.70)	7.58(1.38)	64.21(6.00)
LLM-R^2	**40.63**(3.40)	**6.81**(2.90)	**27.40**(3.29)

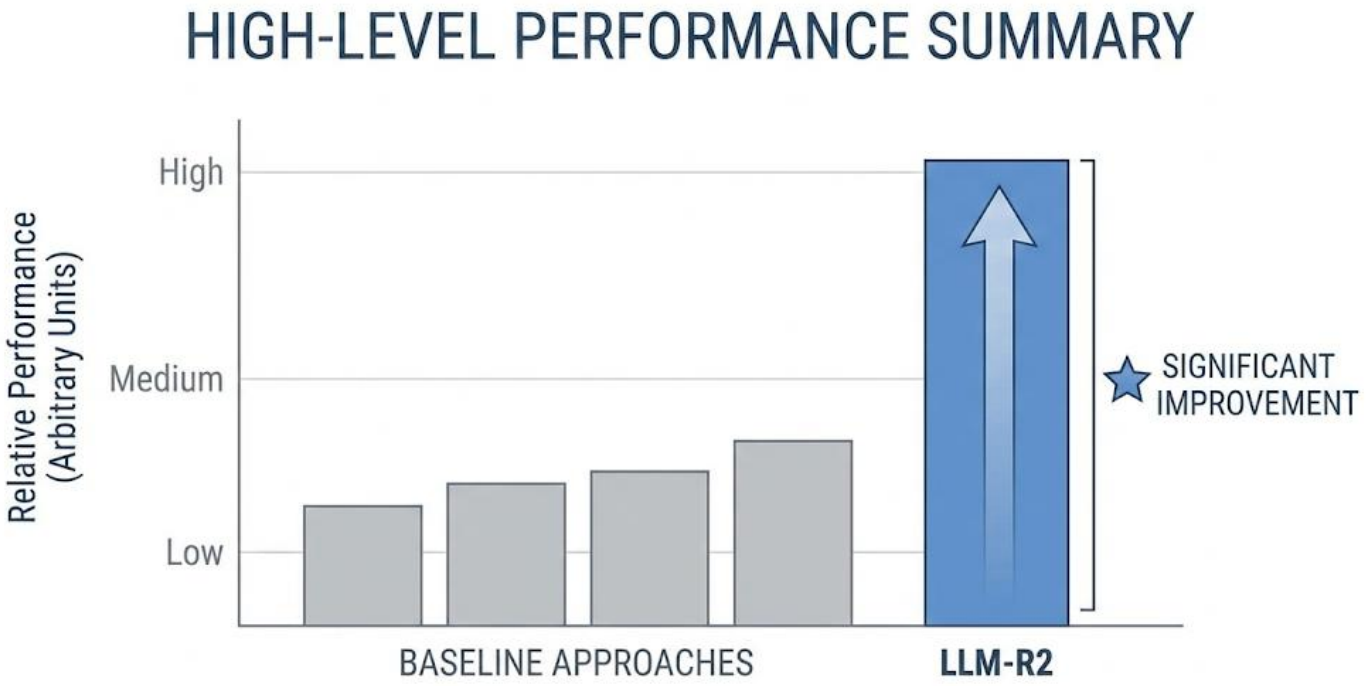
Execution Efficiency Results

- LLM-R2 significantly reduces query execution time across all datasets, outperforming all baseline methods.
- It requires only **52.5% (TPC-H)**, **56.0% (IMDB)**, and **39.8% (DSB)** of the original query time.
- It also outperforms the state-of-the-art baseline (LR), achieving as low as **40.7%** of LR's time on the DSB dataset.



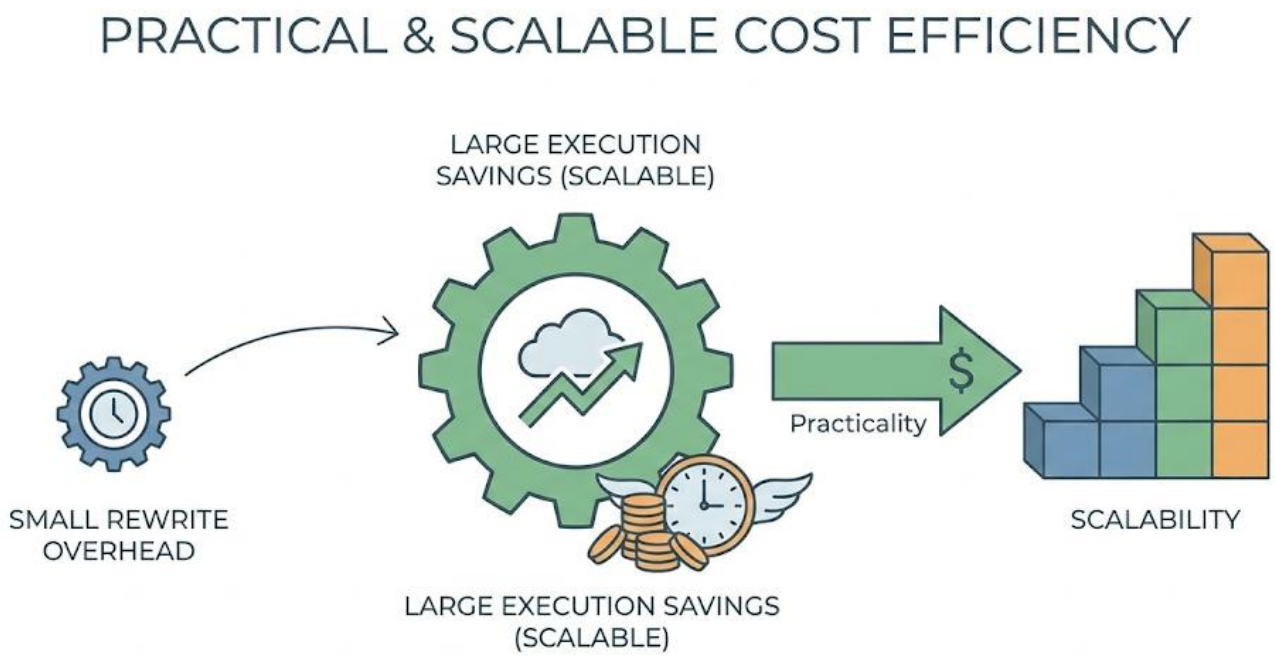
Computational Efficiency and Cost

- LLM-R2 adds a small amount of rewrite latency (2.90 to 3.40 seconds) compared to traditional methods.
- However, for complex queries, the total cost (latency plus execution) remains significantly lower than baselines.
- The average monetary cost per query is less than **0.001 USD**, making it highly cost-efficient.
- Training the model takes less than two hours on a single GPU.



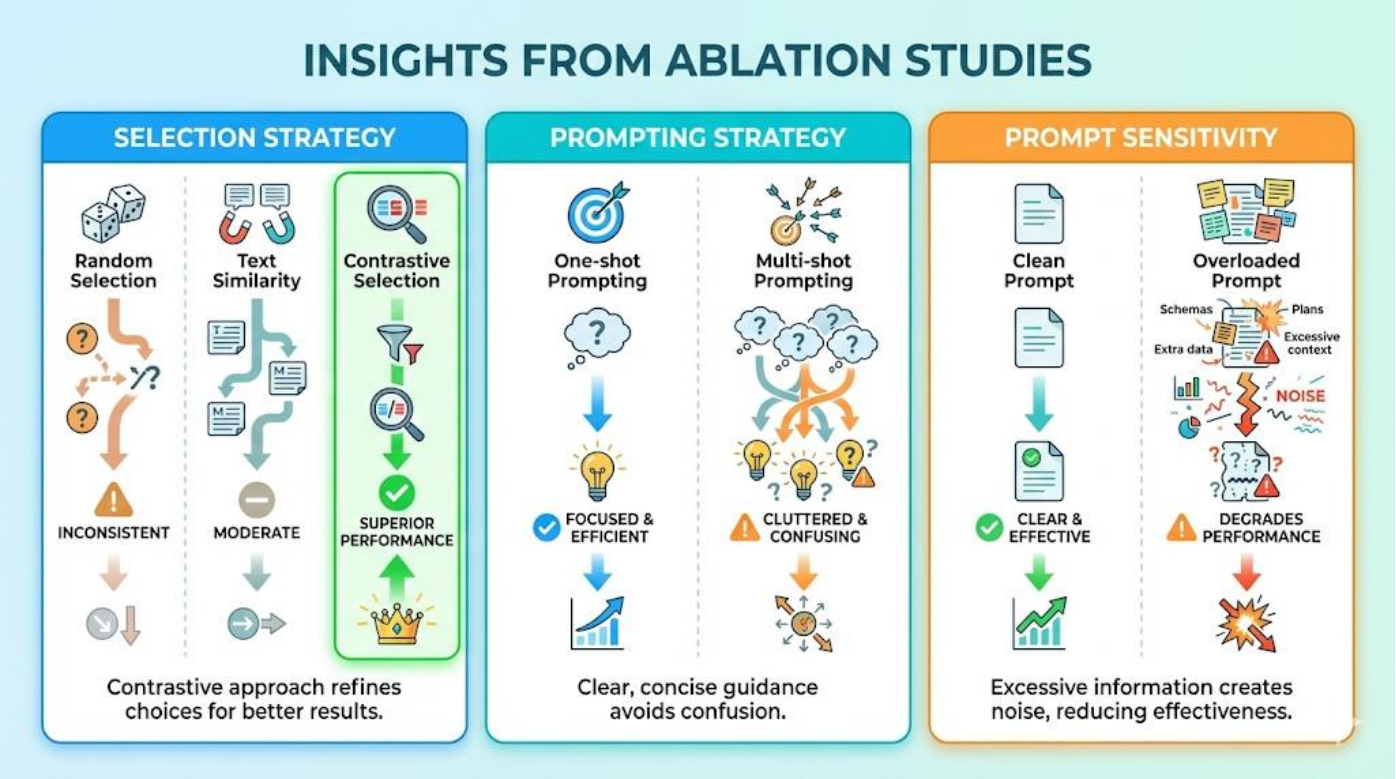
Robustness: Transferability and Scale

- **Transferability:** A model trained on TPC-H was successfully applied to the IMDB dataset, significantly outperforming LLM only.
- **Flexibility:** System efficiency increases consistently as database volume scales from **1GB to 100GB**.
- These results demonstrate the model's ability to generalize to unseen datasets and larger data volumes.



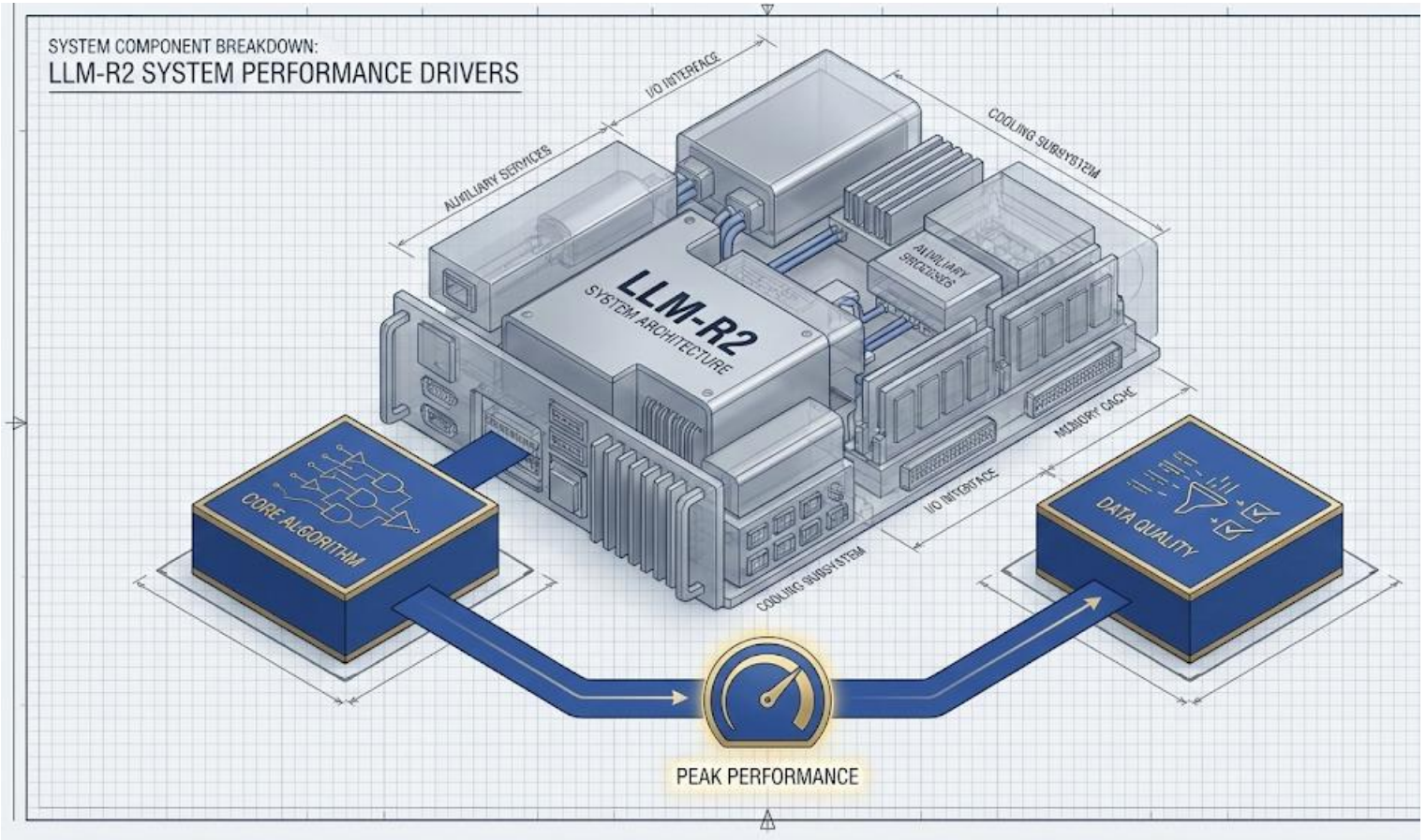
Insights from Ablation Studies

- **Contrastive Selection:** The learned selection model is superior to random, tree-based, or textual similarity approaches.
- **One-shot is Optimal:** Using a single demonstration is more effective than three-shot prompting, which can lead to inconsistent guidance.
- **Prompt Sensitivity:** Adding too much information, such as full schemas or physical plans, can degrade performance as LLMs are sensitive to noise.



Qualitative Case Study

- LLM-R2 discovers rewrite rules that traditional cost-based models (like LR) often miss.
- It avoids "greedy" selection mistakes by recommending a broader spectrum of unique and effective strategies.
- The system effectively transforms complex clauses into efficient joins, proving its practical effectiveness in query optimization.



Conclusion and Future Work

- LLM-R2 is the first framework to integrate LLMs into a rule-based query rewrite pipeline.
- It successfully improves query efficiency across popular datasets while ensuring correctness.
- Future improvements include implementing faster selection algorithms like **Faiss** and fine-tuning models on larger SQL datasets.